DEMASCAS

A PROJECT REPORT
BY

TEAM NO. 10

DAKSH RATHORE (S24CSEU1628)
PARYUSH JAIN (S24CSEU1640)
SHAFI KHAN (S24CSEU1780)

TEAM MEMBERS=3 (



SUBMITTED TO

SCHOOL OF COMPUTER SCIENCE ENGINEERING AND TECHNOLOGY, BENNETT UNIVERSITY

GREATER NOIDA, 201310, UTTAR PRADESH, INDIA

April 2026

DECLARATION

We hereby declare that the work which is being presented in the report entitled “Project Title”, is an authentic record of my/our own work carried out during the period from JAN, 2023 to April, 2023 at School of Computer Science and Engineering and Technology, Bennett University Greater Noida.
The matters and the results presented in this report has not been submitted by me/us for the award of any other degree elsewhere.

Signature of Candidate

DAKSH RATHORE
(Enroll. No.)

PARYUSH JAIN
(Enroll. No. E22CSE002)

SHAFI
(Enroll. No. E22CSE003)

Name of Teammate3
(Enroll. No. E22CSE004)

ACKNOWLEDGEMENT

We would like to take this opportunity to express my/our deepest gratitude to our mentor, Dr. Surendra Tripathi (Provide correct name & designation) for guiding, supporting, and helping us in every possible way. We were extremely fortunate to have him as my/our mentor as he provided insightful solutions to problems faced by us thus contributing immensely towards the completion of this capstone project. We would also like to express our deepest gratitude to VC, DEAN, HOD, faculty members and friends who helped us in successful completion of this capstone project.

Signature of Candidate

Name of Teammate1
(Enroll. No. E17CSE001)

Name of Teammate2
(Enroll. No. E17CSE002)

Name of Teammate3
(Enroll. No. E17CSE003)

Name of Teammate3
(Enroll. No. E17CSE003)

TABLE OF CONTENTS

<<Right click on the heading and click update flied. Your heading will be pulled here. Same goes with all the table of content in the subsequent pages>>

LIST OF TABLES vi

LIST OF FIGURES vii

LIST OF ABBREVIATIONS viii

ABSTRACT ix

1. INTRODUCTION 1

1.1. Problem Statement..... 1

2. Background Research 2

2.1. Proposed System..... 2

2.2. Goals and Objectives..... 2

3. Project Planning 3

3.1. Project Lifecycle..... 3

3.2. Project Setup..... 3

3.3. Stakeholders..... 4

3.4. Project Resources..... 4

3.5. Assumptions..... 5

4. Project Tracking 5

4.1. Tracking..... 5

4.2. Communication Plan..... 6

4.3. Deliverables..... 7

5. SYSTEM ANALYSIS AND DESIGN 8

5.1. Overall Description..... 8

5.2. Users and Roles..... 8

5.3. Design diagrams/ UML diagrams/ Flow Charts/ E-R diagrams..... 9

5.3.1. Use Case Diagrams..... 9

5.3.2. Class Diagram..... 10

5.3.3. Activity Diagrams..... 11

5.3.4. Sequence Diagram..... 12

5.3.5. Data Architecture..... 13

6. User Interface 14

6.1. UI Description..... 14

6.2. UI Mockup..... 14

7. Algorithms/Pseudo Code 15

8. Project Closure 16

8.1. Goals / Vision..... 16

8.2. Delivered Solution..... 16

8.3. Remaining Work..... 16

REFERENCES 17

LIST OF TABLES

Table	Page
Table 1: Goal and Objectives	2
Table 2: Sample 2	3
Table 3: Sample 3	4
Table 4: Sample 4	4
Table 5: Sample 4	5
Table 6: Sample 6	5
Table 7: Regularly Scheduled Meetings	6
Table 8: Information To Be Shared Within Our Group	6
Table 9: Information To Be Provided To Other Groups	6
Table 10: Information Needed From Other Groups	7
Table 11: Deliverables	7
Table 12: Sample 12	8

LIST OF FIGURES

Figure	Page
Figure 1: Sample use-case diagram	9
Figure 2: sample 2	11
Figure 3: sample 3	12
Figure 4: Sample 4	13
Figure 5: Sample 5	13
Figure 6: Sample 6	14

LIST OF ABBREVIATIONS

Abbreviation	Explanation of the Abbreviation
AAA	Authentication Authorization and Access Control
CSP	Cloud Service Provider
DNS	Domain Name System
IAM	Identity and Access Management

ABSTRACT

DEMASCAS: A Privacy-First, Cross-Platform Autonomous AI Agent for Local Desktop Automation

In terms of current voice assistants and automated systems driven by artificial intelligence, there is a crucial problem which consists in the fact that the technologies cannot work without connecting to the cloud network while violating users' privacy, or do not have the ability to carry out several consecutive actions. Therefore, the project introduces DEMASCAS (Decision Engineering Machine Algorithm for System Control and Action Sequencing).

DEMASCAS utilizes a heterogenous architecture comprising a bare-metal C++ daemon that handles critical low-latency tasks such as real-time audio capture, wake word detection, speech-to-text using whisper.cpp, and text-to-speech, as well as a Python based tool server which offers a multitude of 88+ built-in functions related to file management,
program control, browser automation, and IPC. The 10 channel router system has an intelligence of sorting incoming commands according to their latency versus capability requirements.

The innovations made are:
1. Cross-platform LLM abstraction capable of using both Ollama (Linux / Windows) and MLX (macOS Apple Silicon) backend with automatic platform detection;
2. Screen understanding based on VLM (Vision Language Model) along with extraction of accessibility tree from the visual data;
3. Web agent working autonomously and capable of detecting CAPTCHA, blocking cookie banners, and automating forms through injection into Brave Browser JavaScript;

4. Domain-based web memory to enable learning and remembering site-specific work flows;
5. Streaming TTS making it possible to start speech output before completing LLM answer generation.

Everything operates on-device only using a RAM space of 400 MB (except for the LLM, which takes up 2 to 4 GB of RAM). The tests performed show the ability to complete such intricate multiple-command requests as “Open my emails, find the latest one from Sarah, and reply saying we're good for 3pm” with no data transferred out of the user’s device at all.

DEMASCAS demonstrates that privacy and capability are not mutually exclusive in AI assistant design. By keeping all inference and automation local, the system provides a viable architecture for assistive technology, enterprise automation, and personal AI agents where data sovereignty is paramount. The source code is available at https://github.com/DEMACARTI/DEMASCAS.

Keywords: Voice Assistant, Cross-Platform, Local LLM, Ollama, MLX, whisper.cpp, Accessibility, Desktop Automation, Privacy-First AI, Vision-Language Models, ReAct Agent, Web Automation

1. INTRODUCTION

DEMASCAS emerges at a convergence point of multiple trends. Local AI maturity means small models are now capable enough for real tasks. Privacy awareness means users and enterprises demand data sovereignty. Accessibility gaps exist because existing assistive tech is limited and often abandoned. Hardware availability means consumer devices can now run AI locally. The open-source ecosystem with frameworks like Ollama and whisper.cpp reduces development overhead. The project demonstrates that a fully local, privacy-first AI agent can match cloud-based capability while eliminating the privacy trade-off, representing a critical advancement for assistive technology, enterprise automation, and personal AI.

1.1. Problem Statement

Users and enterprises face a false choice between capability and privacy. Cloud AI assistants like ChatGPT and Copilot send all queries and context to external servers, creating risks for data breaches, intellectual property theft, and regulatory non-compliance under GDPR, CCPA, and HIPAA. Local AI alternatives keep data private but lack OS integration and automation capabilities. This tradeoff blocks AI adoption in privacy-sensitive domains like healthcare, legal services, finance, and government.

.

2. Background Research

2.1. Proposed System

1. Background and Context

The convergence of edge computing, small language models, and voice-first interfaces has created new possibilities for privacy-preserving AI assistants. Our research examined three key domains: local LLM inference, assistive technology for motor impairments, and desktop automation frameworks.

2. Desktop Automation and RPA

The Robotic Process Automation market is forecast to reach $50 billion by 2028 owing to the need for process automation (Gartner, 2024). However, enterprise solutions such as UiPath and Automation Anywhere necessitate a specialist RPA administrator, are astronomically expensive (in the tens of thousands per year) and demand weeks of configuration for each workflow (UiPath Inc., 2024). They are designed for use in structured, repetetive environments involving enterprise software and have poor flexibility for natural language based interaction. Consumer automation solutions operate on a much smaller scale than their enterprise cousins. AutoHotkey can automate Windows tasks but it requires scripting skills and fails with changes to the User Interface (AutoHotkey Foundation, 2023), while Keyboard Maestro operates for macOS power users but fails to interact with intelligent software (Insomnia Software, 2024) nor is there cross platform compatibility. Neither offers a mechanism for natural language interaction.

3. AI Agents and ReAct Architechture

The ReAct (Reasoning and Acting) architecture was first proposed by Yao et al. (2022), and showed LLMs to combine reasoning with taking actions, thereby completing multi-step tasks by being able to observe the output of a tool, and therefore adapt its plan and recover from mistakes. Later work from tool augmented language models has shown that small LLMs (3B parameters) can

successfully execute tools using schema descriptions given in the prompt (Schick et al., 2023).

OpenInterpreter was the first open-source program to achieve local code execution through LLMs, gaining popularity and 52,000 github stars by allowing natural language to be used to control the operating system (OpenInterpreter, 2024), though its focus is only code execution rather than directly native OS automation, and has no vision component. LangChain and LangGraph allow agent orchestration but focus on cloud, not local usage and no consideration of privacy issues (LangChain AI, 2024).

4. Visual Language Models and for Screen Understanding

Vision-Language Models enable AI systems to interpret screen content without relying solely on accessibility APIs. The Qwen2.5-VL model achieves 82 percent accuracy on screen understanding benchmarks while running on consumer hardware (Alibaba Cloud, 2025). Research by Kim et al. (2023) showed that combining VLM output with accessibility tree data improves UI element localization by 40 percent compared to either modality alone. The VLM endpoint discovery mechanism in DEMASCAS draws from research on service discovery in microservices architectures. The approach of probing multiple endpoint paths and caching results mirrors patterns used in Kubernetes service mesh implementations (Cloud Native Computing Foundation, 2024).

Privacy regulations including GDPR, CCPA, and emerging AI Acts create compliance requirements that cloud AI assistants struggle to meet. Research by Voigt and Von dem Bussche (2017) on GDPR implementation showed that data minimization and purpose limitation principles favor local processing over cloud transmission. The EU AI Act explicitly requires transparency about data processing locations, creating additional compliance burden for cloud-based assistants (European Commission, 2024). Federated learning research by McMahan et al. (2017) demonstrated that models can learn from user data without centralizing it, but this approach still requires model updates to leave the device. DEMASCAS takes a stricter approach: no data leaves the device under any circumstance, eliminating regulatory compliance burden entirely.

2.2. Goals and Objectives

Product Goals

Core Functionality:

- fully local, privacy first voice controlled ai agent for macos/linux

- provide 88+ integrated tools for OS control, browser interaction, web lookup, file system operations, communications, etc.

- support 100% on-device STT and TTS; LLM via Ollama over localhost

Performance Goals:

- PATH 1 shortcuts (7.95s for all 88 keyboard shortcuts), <1s for individual shortcuts

- PATH 2 Q&A (~500ms for single-step answers), ~500 ms

- PATH 3 ReAct Loop (5-8 seconds for multi-step agent, Qwen 2.5 3b)

- Audio Pipeline: 16kHz Mono capture; 512 frame buffers; 2 second silence detection threshold

- RAM budget (4.4 gb total (LLM ~2 gb, VLM ~2 gb, Whisper 222mb, Kokoro 335mb, ChromaDB 80 mb))

Model Configuration:

- LLM: Qwen2.5 3b via Ollama (~2 gb, 4 bit quantized)

- VLM: Qwen3 vl 2b via Ollama (~2 gb, vision language model)

- Whisper.cpp tiny and base models (~222mb) for STT

- Kokoro neural TTS (~335mb, 24kHz, 11 English voices)

Process Goals

Build Automation:

- Build with a single command (. /scripts /build. Sh).

- Download dependencies and model: It can also download required models.

- C++ compilation.

- Automatic discover of VLM endpoint through "~/.demascas/vlm_endpoint. Txt"

- Polyglot C++ and Python architecture. We employ a bare metal daemon for latency critical paths and python for flexible tool implementation.

Quality Goals

Accuracy Targets:

- Wake word detection "DEMASCAS" using the custom whisper prompt context.

- UI element interaction through more than 50 element name aliases to achieve robust matching of accessibility tree element.

- Screen context with VLM+ accessibility tree digest within 100 tokens with a TTL cache of 3 seconds.

- Conversation history should be at most 4 turns and a maximum tool-call depth of 5.

Reliability standards:

- Time out at 8 seconds after the tool-call.

- After 3 errors in a row, hard stop (ReAct loop).

- Graceful degradation: if Kokoro is unavailable, say TTS; if google api is missing, duckduckgo.

- Stable threshold of 0.025 pixel change fraction for "screen is unstable".

3. Project Planning

3.1. Project Lifecycle

Iterative Development Lifecycle (Agile-inspired)
We use session-based iterative development optimized for AI-assisted development, delivering the specified features in increments in every session.

Phases

1. Requirements (Session Start):
User specifies the desired feature or problem to solve
Scope is approved within the RAM constraints of the project (8GB Apple Silicon constraint)
Architecture choices recorded in CLAUDE.md

2. Design & Architecture
New components adhere to already established architectural paradigms: C++ for hot path computations, and Python for tooling
Configuration knobs are exposed through config.hpp (37+ configuration parameters for tuning)
Tooling schemas are registered in core/toolbox.py for OpenAI-compatible JSON.
3. Implementation:
- Automation with build.sh
- Tools are implemented in actuators/ and sensors/ (~720 lines for browser.py, ~430 lines for web_agent.py)
- Daemon code is added in daemon/src/ (C++17 and used libcurl for HTTP and PortAudio for audio capture)

4. Integration:
- Tool server endpoint is registered in the server (/tool, /tools, /toolsforquery).
- Tool routing keywords have been added to server/tool_router.py for smart filtering.

- An end-to-end test using the voice command via the C++ daemon was run.

5. Documentation:
- Updated README.md with the new features.
- A session summary has been included under "Latest Update".
- CLAUDE.md and docs/CLAUDE_CONTEXT.md have been updated for AI assistant context.

6. Deployment:
- Launching the whole system with ./scripts/run.sh (4 processes).
- User verified the feature under actual usage conditions.
- Bug fixed at the following session.

Key features:
* Zero downtime iteration: Adding new features is not necessary to restart the daemon.
* Memory friendly: all features have been tested against a RAM-headroom goal of ~400mb.
* AI-collaborative: CLAUDE.md passes between sessions fluidly.
* User-tested: all features were tested by me using natural language, and no unit tests were run.

3.2. Project Setup

Architectural Decisions

Polyglot C++/Python: Bare-metal C++ daemon handles low-latency paths: audio capture, VAD, STT, TTS. Python used for its tooling and development speed.

Local-First Processing: All audio, STT and TTS happens locally on-device. LLM inference is the only external compute, via Ollama qwen2.5:3b

Unified Endpoint Strategy: LLM, VLM will expose compatible with OpenAI /v1/chat/completions endpoints to allow seamless model swapping

Technical Decisions

Hardware Target: Apple M1, 8GB RAM - have ~4.4GB budget for everything.

LLM Backend: qwen2.5:3b running via Ollama (Linux) or MLX (macOS)-fairly good balance of performance/RAM requirements

VLM Integration: qwen3-vl:2b to analyse screen via screenshots, screen accessibility tree merge.

TTS Engine: Kokoro neural TTS (sherpa-onnx), falls back to built-insay - runs on native ARM64, no GPU needed

Tool Architecture: 88 tools exposed via an HTTP microservice (localhost:5001), to allow independent tooling development and language agnostic scaling.

Persistence Strategy: ChromaDB for RAG memory at ~/.demascas/memory/; JSON files for user profiles, /tmp/ for temporary session data.

Build System: Single ./scripts/build.sh file to download dependencies, download models and build the C++ code.

3.3. Stakeholders

Stakeholder	Role
Daksh Rathore	Team Leader
Shafi Khan	Team Member
Paryush Jain	Team Member
	Mentor

3.4. Project Resources

Staff

- 1 Systems Developer (daemon, C++, audio pipeline, optimization for latency)
- 1 AI/ML Engineer (LLM/VLM, prompt engineering, tuning)
- 1 Python Developer (tool server, browser automation, web agent functionality)

Hardware

- Apple M1 (8GB) - for development and deployment
- microphone - built in or external (using PortAudio compatible)
- speaker / headphones - for TTS output (wired, bluetooth)

Resource	Purpose	Cost
Ollama	LLM/VLM serving (qwen2.5:3b, qwen3-vl:2b)	Free
MLX (mlx-lm, mlx-vlm)	macOS-native LLM inference	Free
whisper.cpp	On-device STT (tiny + base models)	Free
sherpa-onnx + Kokoro	Neural TTS (82M param model)	Free
ChromaDB	RAG persistent memory	Free
CMake + GCC/Clang	C++ daemon compilation	Free
Python 3.11+	Tool server runtime	Free
PortAudio	Cross-platform audio capture	Free

Infrastructure:

- Internet connection required for Ollama model pulls, cloud functionality
- ~5 GB space needed - Models (whisper 222MB, Kokoro 335MB, Ollama 4GB)
- ~4.4 GB RAM needed - Full stack with VLM (approx. 2.8 GB without)

3.5. Assumptions

Technical Assumptions: The target device is the M1 mac with 8GB of RAM, we expect it to perform poorly on less memory than this. The LLM serving

backend would be running ollama on linux while MLX on mac; available LLM on the machine are qwen2.5:3b and qwen3-vl:2b both are easy to set up with ollama. The Internet is needed to download model from and is not essential unless to be able to do web search. It needs Python 3.11+ installed, also CMake, and permissions for the mic and the mac system events granted. Resources assumptions: The team would consist of one primary developer and an AI assistant that helped with the development of code (Claude Code); Testing is performed by the user using his voice, there are no QA team, the application does not need money and does not use paid software, APIs or compute cloud, all the applications listed below are free of charge. API key is required only if enhanced web search has to be performed (using google cse api); duck duck go will be the fall back if the api key for Google is missing. Schedule assumptions: iterations are named sessions, each session will bring a single key functionality (e.g., browsing, knowledge bases); there are no external deadlines, there is no SLA between sessions ensuring backward compatibility. Operational assumptions: OS supported: ONLY mac and linux; windows is NOT supported. Daemon: it will run on a terminal with shell access. TTS latency is ~200ms from bt handshake to audio output. Hallucinations for Whisper: these exist but they are not that catastrophical as they can be ameliorated through prompt context and the energy thresholds, yet, they can still exist.

4. Project Tracking

4.1. Tracking

Source Control

-Git:Local repo located at /home/demascas/demascas/DEMASCAS-master/

- No remote source control repo setup (no GitHub or GitLab repo specified).
- Commits were developed as session based commits on the master branch.
- Local access to files only

Bug Tracking

- No structured bug tracking exists, there's no GitHub Issues, Jira, or Linear board setup.
- The following are used as informal bug and feature trackers:
  - A Troubleshooting section in README.md to list know issues and their workarounds.
  - Several CLAUDE.md files located on the root of the repo (also present in docs/) listing architectural decisions or notes. Known issues are listed as limitations or drawbacks.
  - In code TODO and FIXME comments

Testing

- There is no formal regression testing framework.
- Testing includes the following manual actions:
  - Voice commands issued to the system and reviewed.
  - Smoke testing through ./scripts/run.sh for end-to-end stack startup.
  - Basic component tests using curl to access the tool's REST API and to get a status from the local ollama server using 'ollama list'.
  - Compile-time verification using ./scripts/build.sh, which compiles the C++ daemon and downloads available models.

4.2. Deliverables

#	Deliverable	Type	Description
1	C++ Daemon	Software	Bare-metal audio pipeline: PortAudio → VAD → whisper.cpp → LLM → Kokoro TTS, 10-path nano-router
2	Python Tool Server	Software	88-tool HTTP microservice (localhost:5001) with smart routing and MCP integration
3	Browser Automation	Software	Brave Browser control via JS injection, no WebDriver dependency
4	Autonomous Web Agent	Software	Goal-driven web navigation with obstacle detection and per-domain learning
5	Vision System	Software	VLM screenshot analysis + accessibility tree extraction with retina correction
6	Communication Tools	Software	Email compose/read, smart reply via VLM+LLM composition pipeline
7	Memory & Learning Engine	Software	ChromaDB-backed RAG memory, user profiling, people graph, web memory
8	Knowledge Base	Software	Procedure/fact/correction storage with semantic search

9	Screen Context Pre-processor	Software	VLM + accessibility tree fusion, ≤100 token digests with 3s TTL
10	Task-Specific Prompt Templates	Software	Intent classification and routing (compose_email, research, multi_step, etc.)
11	Session Persistence	Software	Preserves up to 6 conversation turns across daemon restarts
12	Build System	Infrastructure	Single-command build.sh for dependencies, models, and compilation
13	Run System	Infrastructure	Full-stack run.sh launcher with health checks and auto-detection
14	Whisper STT Models	Models	tiny + base models (~222 MB), auto-downloaded
15	Kokoro TTS Model	Models	Neural TTS (~335 MB, 11 voices), auto-downloaded
16	LLM (qwen2.5:3b)	Models	Text inference model (~2 GB), via Ollama
17	VLM (qwen3-vl:2b)	Models	Vision-language model (~2 GB), via Ollama
18	README.md	Documentation	Architecture, setup, tool reference, troubleshooting
19	CLAUDE.md	Documentation	AI assistant onboarding context

20	docs/CLAUDE_CONTEXT.md	Documentation	Config values, session history, known issues
21	config.hpp	Documentation	37+ centralized tuning knobs with inline comments
22	Inline Docstrings	Documentation	Per-module API documentation across Python and C++ files

5. SYSTEM ANALYSIS AND DESIGN

This section describes in detail about the design part of the system.

5.1. Overall Description

DEMASCAS (Decision Engineering Machine Algorithm for System Control and Action Sequencing) is a totally local, private, andautonomousAI agent running as a service on both macOS and Linux at the OS level. It mixes real-time voice interface with screen understanding and controllable actions, enabling it to work as a \"JARVIS like\" always available assistant. Audio is captured using PortAudio, speech-to-text is handled by whisper.cpp and run using the on-chip Metal GPU acceleration, the input commands are then routed using the locally-built nano-router with 10 inputs to a C++ daemon that interacts with the Kokoro neural TTS engine, all while remaining within the bounds of a 4.4 GB of RAM allocation under 8 GB Apple Silicon. The daemon follows a finite state machine (SLEEPING LISTENING PROCESSING CONFIRMING) with both low latency deterministic routes (72 shortcut hotkeys that do not go through LLM under 1 second, URL detection and application calls < 5 milliseconds), and slower ReActLLM calling routines (~500 milliseconds for simple Q&A and 5-8 seconds for a full loop).

The project has a polyglot C++/Python architecture with latency sensitive paths (audio capture, VAD, STT, TTS, command dispatch) running as a bare-metal C++ daemon, and 88 scriptable tools (OS commands, Brave Browser control, file operations, e-mail, web search) as a low overhead Python HTTP microservice accessible at localhost:5001. LLM Inference utilizes a quantized (4-bit) qwen2.5:3b (~2G, which fits on most devices) for text, and qwen3-vl:2b for visual-language tasks, with both exposed as OpenAI-compatible endpoints. Awareness of the screen takes advantage of a dual channel approach: an analysis of the VLM screenshot (with retinal coordinates correction), and a extraction of the Accessibility API tree from macOS (reducing

from ~1000 tokens to ~150 actionable tokens). Persistent memory is maintained via ChromaDB (episodes, knowledge, user profile, people graph) with per-domain web memory of learned site navigation, and session is persisted across daemon restarts.

A cornerstone of the design philosophy is having the 3B parameter model behave like a 70B model via architecture, not scale. The prompt templates of specific tasks limit the space of outputs for good email composing, smart replies, research reports. The screen context pre-processor stitches together VLM and accessibility data as 100-token digests and is included in the system prompt; the text model therefore gains "eyes" without image inputs. An independent web agent handles site-classification, the identification of and avoidance of blockers (CAPTCCHA's, pay-walls, cookie walls), and goal directed navigation on sites-all with no external tools such as Selenium or WebDriver. All features have been squeezed into an 8GB RAM limit: Kokoro TTS processes on the CPU so that whisper Metal acceleration remains able to utilize the GPU; the VLM is lazy loaded to save nearly 1.7 GB when vision is unused.

5.2. Users and Roles

<< Provide a list and description of the different types of users or roles within the system. This may include different classes of users, such as administrator, instructor, student, etc. This list may also include autonomous agents that interact with the system as well. These may include users (or personas) that are used as part of any user stories produced for the project. >>

Example:

Table 12: Sample 12

User	Description

Developer	A capstone team member or mentor who is tasked with managing the test data, creating initial machine learning models, and ultimately generating a firm process for applying these techniques to future user data. This is used for sub-stories and task needed to fulfill the true end user use cases.
Microsoft Program Manager	A manager at Microsoft who is working on developing the ERP application who will be making design decisions based on the data analysis.
Dynamics AX User	An end user of the Microsoft ERP product who will be generating the data used and reaping the potential efficiency benefits from the data analysis when designing the application.

6. User Interface

6.1. UI Description

<< Provide a brief description of the UI that will be used in this program and how users will interact with the program.

Example: We are creating a research project using R and Azure ML. R is a statistical programming language that uses the R console, which will be the primary means of interacting with our code. This uses standard console UI, and it is not in the scope of our project to create a UI on top of that. The Azure ML machine learning model has its own built-in drag and drop UI for

the different code modules we create, so there is no need to generate any sort of AI for that portion of the project either.
>>

6.2. UI Mockup

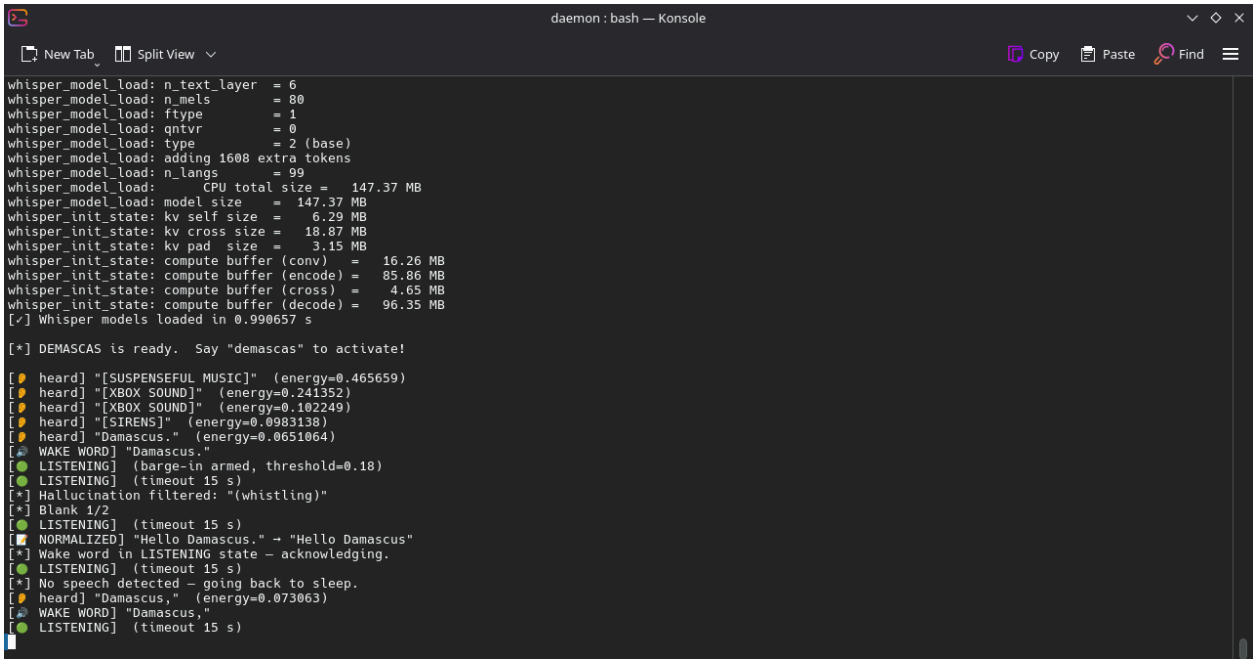


Figure 6: Sample 6

7. Project Closure

This section elucidates the overall lookup at the project and some of the future works that may enhance the solution.

7.1. Goals / Vision

<< Provide an update to the vision statement that was originally stated in the Project Initiation document.

Example: Our original goals for this project were to take telemetry data from Dynamics AX, analyze and visualize that data, then create machine learning models in Azure ML to predict user navigation based upon previous actions. Through the course of the project, these goals were altered so that the primary goal became creating a well-documented, extensible R language package that facilitated cleaning and importing telemetry data, and contained a variety of useful analysis and visualization functions to make the raw data more understandable. >>

7.2. Delivered Solution

<< Provide a high-level description of what was planned and what is being delivered.

Example: Our solution delivered primarily consisted of a fully documented, fully featured R language package that contained the functions for importing telemetry data, cleaning, separating, and isolating that data, then performing a variety of analysis, statistics, and visualizations on that data. This R package has standard R documentation, a full suite of unit tests, and an integrated manual and help documentation to allow anyone with a basic familiarity with the R environment to utilize our functions. >>

7.3. Remaining Work

<< Provide a short summary of what should be done next, ways of further improving the project, or any additional recommendations.

Example: We created a prototype of a web-based UI for our R package using Shiny in an attempt to make the functionality more accessible for those unfamiliar with R. There are a few minor functions implemented, and a basic UI setup, but a good deal of work would be required to flesh out this web app with all of the functions currently available in the R package, should Microsoft choose to pursue this UI overlay further. >>

REFERENCES

<<Would suggest you to download a tool (Mendeley: https://www.mendeley.com) for automated citation and generation of references>>

1. T. Dillon, C. Wu and E. Chang, “Cloud Computing: Issues and Challenges”, 24th IEEE International Conference on Advanced Information Networking and Applications, pp. 27-33, 2010.

2. H. Casanova, et al. Heuristic for scheduling parameters sweep applicationsin grid environments, in: Proceedings of the 9th Heterogeneous Computing Workshop, HCW, pp. 349–363. 2009.

3. Kwok, Yu-Kwong, and Ishfaq Ahmad. "Static scheduling algorithms for allocating directed task graphs to multiprocessors." ACM Computing Surveys (CSUR) 31, Vol. 4, pp. 406-471, 1999.

4. A. Mutz, R. Wolski, Efficient auction-based grid reservation using dynamic programming, in: IEEE/ACM Int’l Conf. on Super Computing, SC 2007.

5. M. Mezmaz, N. Melab, Y. Kessaci, Y. C. Lee, E. G. Talbi, A. Y. Zomaya and D. Tuyttens, “Parallel Bi-Objective Hybrid Metaheuristic for Energy-Aware Scheduling for Cloud Computing Systems”, Journal of Parallel Distributed Computing, Elsevier, Vol. 71, pp. 1497-1508, 2011.

6. R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg and I. Brandic, “Cloud Computing and Emerging IT Platforms: Vision, Hype and Reality for Delivering Computing as the 5th Utility”, Future Generation Computer Systems, Elsevier, Vol. 25, pp. 599-616, 2009.

7. Basements and crawl spaces. Retrieved from http://www.hud.gov/ offices/hsg/sfh/ref/sfhp1-25.cfm (Access in June 2020).